



UNINASSAU

LIGAÇÃO DINÂMICA (DYNAMIC BINDING) E TRATAMENTO DE EXCEÇÕES.

Análise e Desenvolvimento de Sistemas
Programação Orientada à Objetos e Estruturas de dados
Profº Eng. Esp. Lindenberg Andrade

Ligação Dinâmica

LIGAÇÃO DINÂMICA

Ligação dinâmica é quando a chamada de um método é resolvida em tempo de execução.

Permite que o polimorfismo funcione corretamente.

Em Java:

Métodos não estáticos utilizam ligação dinâmica.

Métodos estáticos utilizam ligação estática (compile-time).

- Exemplo:

```
class Animal {  
    void som() {  
        System.out.println("Animal faz um som");  
    }  
}  
  
class Cachorro extends Animal {  
    void som() {  
        System.out.println("Cachorro late");  
    }  
}  
  
public class Teste {  
    public static void main(String[] args) {  
        Animal a = new Cachorro(); // referência de Animal para objeto Cachorro  
        a.som(); // ligação dinâmica: imprime "Cachorro late"  
    }  
}
```

O método chamado depende do **objeto real (Cachorro)** e não da referência (**Animal**).

NÃO ESTÁTICOS E LIGAÇÃO DINÂMICA

- Métodos não estáticos são ligados ao objeto em tempo de execução.
- O compilador não decide qual método será chamado; isso acontece em tempo de execução (runtime).
- Permite polimorfismo, ou seja, o mesmo código pode se comportar de forma diferente dependendo do objeto real.

- Exemplo:

```
class Animal {  
    void som() {  
        System.out.println("Animal faz som");  
    }  
}  
  
class Gato extends Animal {  
    void som() {  
        System.out.println("Gato mia");  
    }  
}  
  
public class Teste {  
    public static void main(String[] args) {  
        Animal a = new Gato();  
        a.som(); // Ligação dinâmica: imprime "Gato mia"  
    }  
}
```

Mesmo que a referência seja **Animal**, o método chamado é o do objeto real (**Gato**).

MÉTODOS ESTÁTICOS E LIGAÇÃO ESTÁTICA

- Métodos estáticos pertencem à classe, não ao objeto.
- O compilador decide em tempo de compilação qual método será chamado → ligação estática.
- Polimorfismo não se aplica a métodos estáticos.

- Exemplo:

```
class Animal {  
    static void info() {  
        System.out.println("Animal info");  
    }  
}  
  
class Gato extends Animal {  
    static void info() {  
        System.out.println("Gato info");  
    }  
}  
  
public class Teste {  
    public static void main(String[] args) {  
        Animal a = new Gato();  
        a.info(); // Ligação estática: imprime "Animal info"  
    }  
}
```

Mesmo que a referência seja **Gato**, o método chamado é o da classe da referência (**Animal**).

BENEFÍCIOS DA LIGAÇÃO DINÂMICA

- **Permite polimorfismo**

- Com ligação dinâmica, você pode usar uma referência de uma classe pai para apontar para objetos de diferentes subclasses.
- O método chamado será o da classe real do objeto, permitindo que o mesmo código se comporte de formas diferentes.
- Exemplo:

```
Animal a1 = new Cachorro();  
Animal a2 = new Gato();  
a1.som(); // Cachorro late  
a2.som(); // Gato mia
```

Aqui, o mesmo método **som()** é chamado, mas o comportamento muda conforme o objeto.

- **Facilita extensibilidade e manutenção do código**

- Você pode adicionar novas subclasses sem precisar mudar o código que usa a classe pai.
- Exemplo: se criar uma nova classe Passaro extends Animal, o mesmo código a.som() funcionará para instâncias de Passaro sem alterações.

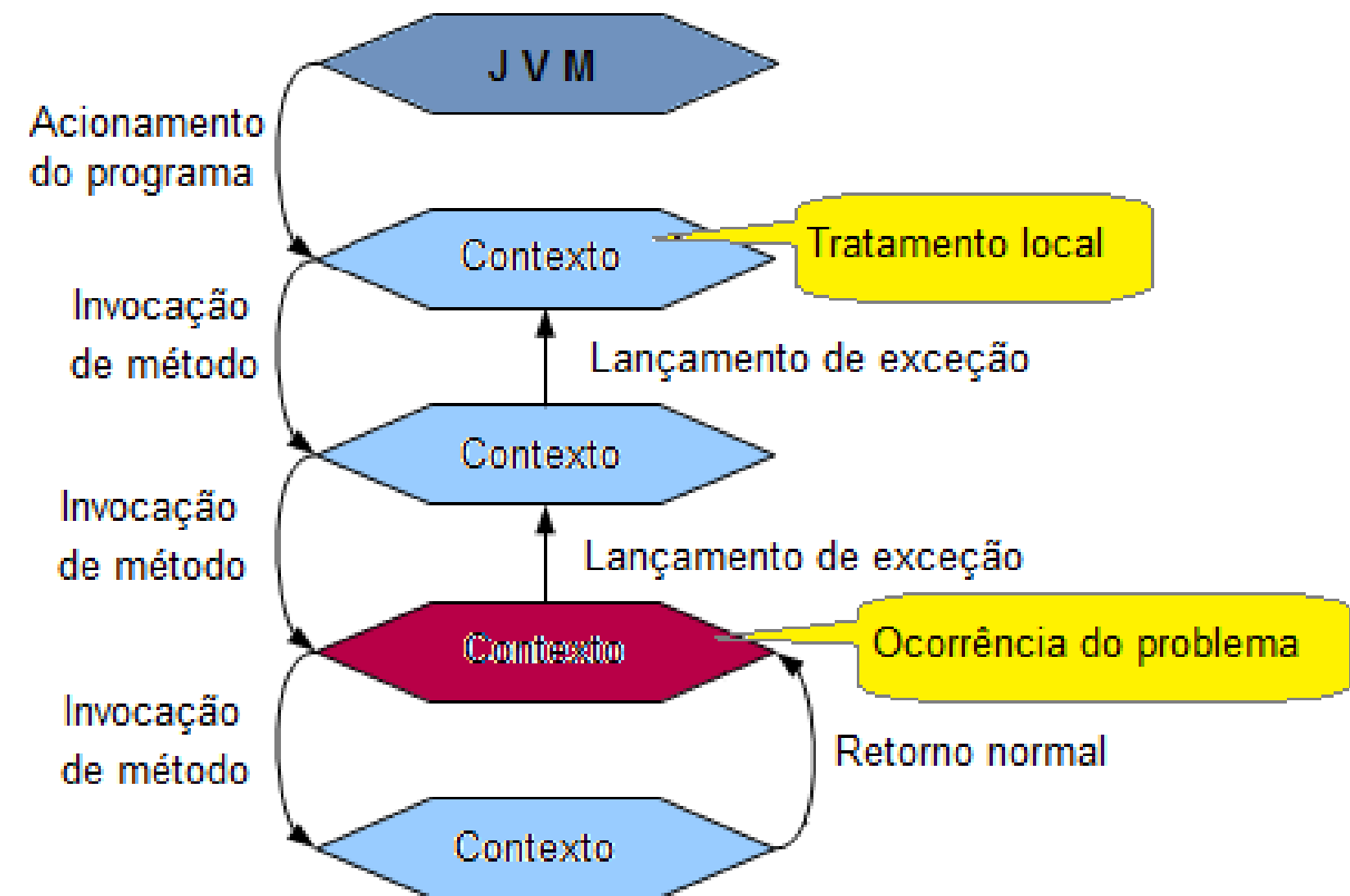
- **Permite reutilização de código**

- O código que chama os métodos não precisa conhecer todas as subclasses, apenas a classe base.
- Isso reduz duplicação de código e melhora a organização do sistema.

Tratamento de Exceções

TRATAMENTO DE EXCEÇÕES

- **Exceção:** evento que ocorre durante a execução do programa, interrompendo o fluxo normal.
- **Objetivo:** capturar e tratar erros, evitando que o programa termine abruptamente.
- **Tipos principais em Java:**
 - **Checked:** devem ser tratadas ou declaradas (**IOException, SQLException**)
 - **Unchecked:** não obrigatórias (**ArithmeticException, NullPointerException**)
 - **Errors:** problemas graves que não devem ser tratados (**OutOfMemoryError**)



CHECKED EXCEPTIONS

- São exceções que o compilador obriga o programador a tratar ou declarar.
- Normalmente representam erros externos ao programa, como problemas de I/O ou acesso a banco de dados.
- Exemplos comuns:
 - **IOException** → erro ao ler/escrever arquivos.
 - **SQLException** → erro ao acessar banco de dados.

- Exemplo:

```
import java.io.File;
import java.io.FileReader;
import java.io.IOException;

public class Teste {
    public static void main(String[] args) {
        try {
            FileReader file = new FileReader("arquivo.txt");
        } catch (IOException e) {
            System.out.println("Arquivo não encontrado!");
        }
    }
}
```

O **try-catch** é necessário para tratar a **IOException**.

UNCHECKED EXCEPTIONS

- Também chamadas de **runtime exceptions**.
- O compilador não obriga o tratamento, mas podem ocorrer durante a execução.
- Normalmente representam erros de programação ou condições inesperadas.
- Exemplos comuns:
 - **ArithmeticException** → divisão por zero.
 - **NullPointerException** → tentativa de acessar um objeto nulo.

- Exemplo:

```
public class Teste {  
    public static void main(String[] args) {  
        int x = 10 / 0; // ArithmeticException  
    }  
}
```

O código compila, mas lança exceção em tempo de execução.

ARITHMETICEXCEPTION

- É uma **Unchecked Exception**.
- Ocorre quando **há operações aritméticas inválidas**, como **divisão por zero**.
- Exemplo:

```
public class TesteArithmetic {  
    public static void main(String[] args) {  
        int x = 10;  
        int y = 0;  
        int resultado = x / y; // Gera ArithmeticException  
        System.out.println(resultado);  
    }  
}
```

- Saída esperada:

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
```

- Explicação:
 - A exceção ocorre **em tempo de execução**, não é necessária declaração com **throws**.
 - Pode ser tratada com **try-catch** se desejado.

NULLPOINTEREXCEPTION

- É uma **Unchecked Exception**.
- Ocorre quando o programa tenta **acessar ou manipular um objeto que está nulo**.

- Exemplo:

```
public class TesteNullPointer {  
    public static void main(String[] args) {  
        String texto = null;  
        System.out.println(texto.length()); // Gera NullPointerException  
    }  
}
```

- Saída esperada:

```
Exception in thread "main" java.lang.NullPointerException
```

- Explicação:
 - A exceção ocorre em tempo de execução.
 - Sempre que houver referência nula, é necessário verificar antes de acessar métodos ou atributos.

ERRORS

- Representam problemas graves do sistema que normalmente não devem ser tratados pelo programa.
- Diferente de exceções, não devem ser capturadas, pois indicam falhas do ambiente de execução.
- Exemplos comuns:
 - **OutOfMemoryError** → falta de memória na JVM.
 - **StackOverflowError** → estouro de pilha.

- Exemplo:

```
public class Teste {  
    public static void recursaoInfinita() {  
        recursaoInfinita();  
    }  
    public static void main(String[] args) {  
        recursaoInfinita(); // StackOverflowError  
    }  
}
```

Tentar capturar **Error** raramente faz sentido; geralmente o programa precisa ser encerrado.

ESTRUTURAS DE TRATAMENTO

- **try { ... }** → bloco de código que pode gerar exceção
- **catch (TipoExcecao e) { ... }** → bloco para tratar a exceção
- **finally { ... }** → bloco que sempre é executado
- **throw** → lança uma exceção

EXEMPLO DE TRY-CATCH

```
public class TesteExcecao {  
    public static void main(String[] args) {  
        try {  
            int resultado = 10 / 0; // causa ArithmeticException  
            System.out.println(resultado);  
        } catch (ArithmeticException e) {  
            System.out.println("Erro: divisão por zero!");  
        } finally {  
            System.out.println("Bloco finally executado");  
        }  
    }  
}
```

Saída esperada:

```
Erro: divisão por zero!  
Bloco finally executado
```

EXEMPLO DE THROW E THROWS

```
public class Validacao {  
    void verificarIdade(int idade) throws Exception {  
        if (idade < 18) {  
            throw new Exception("Idade menor que 18");  
        } else {  
            System.out.println("Idade válida");  
        }  
    }  
  
    public static void main(String[] args) {  
        Validacao v = new Validacao();  
        try {  
            v.verificarIdade(16);  
        } catch (Exception e) {  
            System.out.println(e.getMessage());  
        }  
    }  
}
```

throws declara a exceção, throw lança a exceção.

Análise passo a passo

- Um objeto **Validacao** é criado.
- O método **verificarIdade(16)** é chamado.
- Dentro do método, **idade < 18** é **verdadeiro**.
- O código executa **throw new Exception("Idade menor que 18")**.
- A exceção é **capturada** no bloco **catch**.
- **e.getMessage()** imprime a mensagem da exceção.

Saída esperada: **Idade menor que 18**

Se chamássemos verificarIdade(20), a saída seria:

Idade válida

EXEMPLO DE FINALLY

- O bloco **finally** é executado **sempre**, independentemente de ocorrer ou não uma exceção.
- Geralmente usado para **liberar recursos**, como arquivos, conexões ou streams.

```
public class TesteFinally {  
    public static void main(String[] args) {  
        try {  
            int resultado = 10 / 0; // Gera ArithmeticException  
            System.out.println(resultado);  
        } catch (ArithmeticException e) {  
            System.out.println("Erro: divisão por zero!");  
        } finally {  
            System.out.println("Bloco finally executado");  
        }  
    }  
}
```

Saída esperada:

```
Erro: divisão por zero!  
Bloco finally executado
```

- O **catch** captura a exceção.
- O **finally sempre** é executado, mesmo após o tratamento do erro.
- Ideal para **liberar recursos** ou realizar operações que devem acontecer independentemente do sucesso ou falha.

BOAS PRÁTICAS NO TRATAMENTO DE EXCEÇÕES

- Tratar apenas **exceções que podem ser resolvidas**.
- Evitar **catch(Exception e)** genérico sem tratamento.
- Utilizar **finally** para liberar recursos (arquivos, conexões).
- Evitar usar exceções para controle de fluxo.

Exercícios Propostos

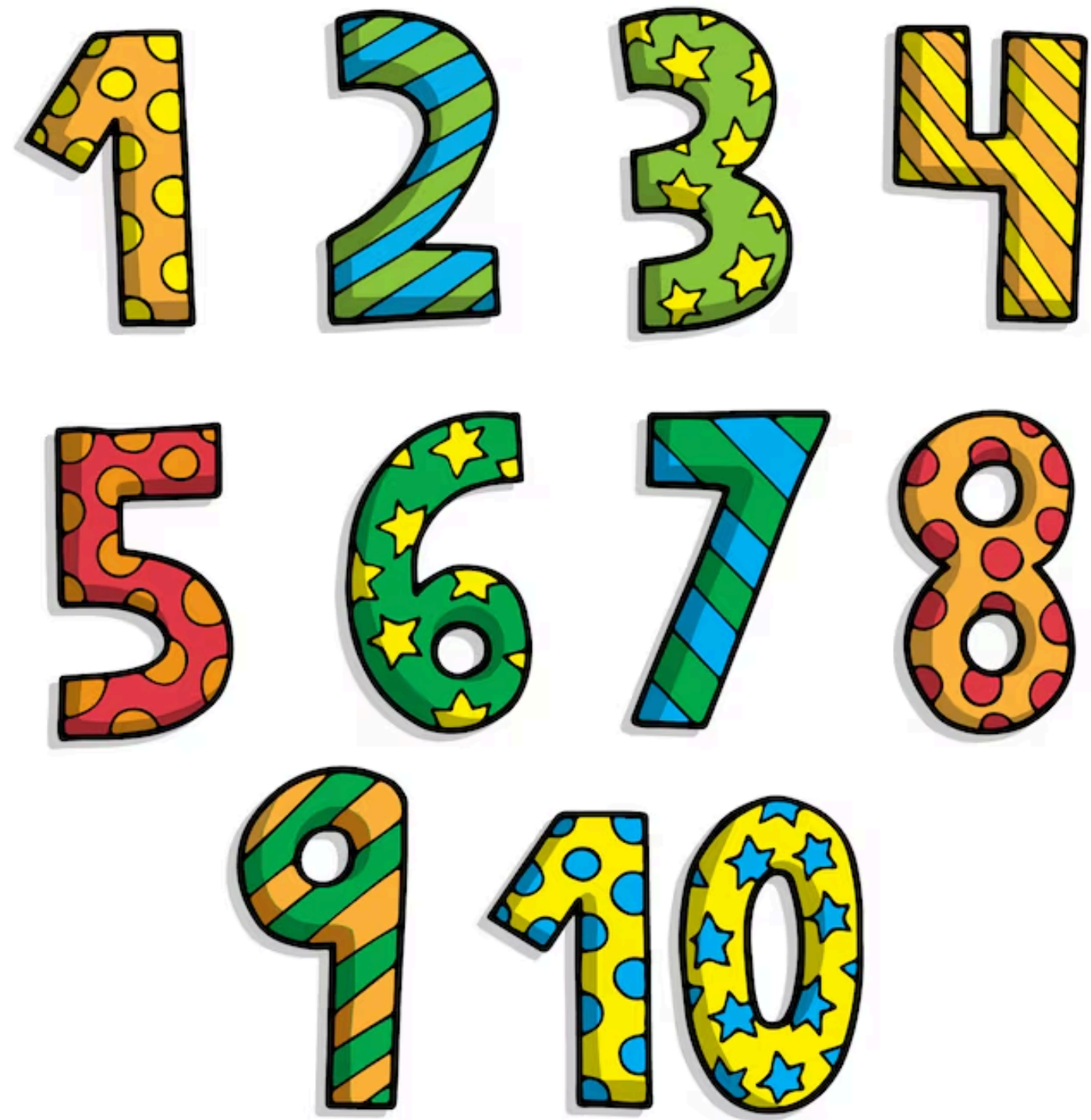
EXERCÍCIO 1: VEÍCULO

- Crie uma classe **Veiculo** e subclasses **Carro** e **Moto**.
- Adicione método **mover()**.
- Crie referências do tipo **Veiculo** apontando para objetos **Carro** e **Moto**.
- Demonstre ligação dinâmica.



EXERCÍCIO 2: ARITHMETICEXCEPTION E INPUTMISMATCHEXCEPTION

- Escreva um programa que:
- Solicite ao usuário um número inteiro.
- Faça a divisão de 100 pelo número fornecido.
- Trate **ArithmeticException** e **InputMismatchException**.



EXERCÍCIO 3: ARQUIVO

- Crie um método **abrirArquivo(String nome)** que lance uma exceção se o arquivo não existir.
- Use **try-catch** para capturar e exibir a mensagem de erro.



REFERÊNCIAS

1. BARNES, David J.; KÖLLING, Michael. Programação orientada a objetos com Java: uma introdução prática usando o BlueJ. 4. ed. São Paulo: Pearson Prentice Hall, 2008.
2. SANTOS, Rafael. Introdução à Programação Orientada a Objetos usando Java. 2. ed. Rio de Janeiro: Elsevier, 2013. 336 p.
3. SERSON, Roberto Rubinstein. Programação orientada a objetos com Java 6 – curso universitário. 1. ed. Rio de Janeiro: Brasport, jun. 2009. 492 p.
4. PREISS, Bruno R. Estruturas de dados e algoritmos: padrões de projetos orientados a objetos com Java. Rio de Janeiro: Campus, 2001. 566 p.
5. Deitel, P. & Deitel, H. Java: Como Programar. 11ª edição, Pearson, 2020.
6. Horstmann, C. Core Java Volume I – Fundamentals. 11ª edição, Prentice Hall, 2019.
7. Oracle Docs: <https://docs.oracle.com/javase/tutorial/>

2001: Uma Odisseia no Espaço (1968)



Considerado por muitos como o melhor e mais importante filme de ficção científica da história, 2001: Uma Odisseia no Espaço é dirigido por Stanley Kubrick e explora diferentes temas, como tecnologia, vida extraterrestre, viagem espacial, evolução humana e, claro, inteligência artificial. A história acompanha uma expedição de astronautas em busca por evidências de que o ser humano não está sozinho. A obra cinematográfica, lançada em 1968, ganhou o Oscar de Efeitos Visuais e foi indicada também às categorias de Diretor, Roteiro Original e Direção de Arte.

2001 é um verdadeiro marco para a história do cinema e está disponível no catálogo do HBO Max. O indiscutível impacto da história de Kubrick (co-escrita por Arthur C. Clarke) é representado nas notas dos sites de crítica. No IMDb, a produção foi avaliada em 8.3, já no Metacritic teve 84 entre a crítica especializada e 8.2 entre o público. No Rotten Tomatoes, a aprovação foi de 91% e 89%, respectivamente.

Dúvidas?



Profº Lindenberg Andrade
E-mail: linndemberg1@gmail.com



Additional contacts via QR code